TECHNICAL CASE STUDY

Resolving Overlapping VPC CIDRs

A Transit Gateway and Sophos Firewall NAT Architecture for Enterprise Cloud Environments

■ Domain: Cloud Networking ■ Platform: AWS ■ Technology: Sophos NGFW / Transit Gateway

Executive Summary

This document presents a detailed technical case study of a cloud network architecture engagement undertaken to resolve a critical connectivity challenge: enabling secure, routed communication between Amazon Web Services (AWS) Virtual Private Clouds (VPCs) whose address spaces were **irreconcilably overlapping**. The scenario is representative of a class of problems that arises frequently in enterprise cloud environments, particularly following mergers and acquisitions, multi-team cloud adoption without centralized governance, or integration with third-party infrastructure.

The primary business driver was the deployment of a Sophos Zero Trust Network Access (ZTNA) solution across the organization's cloud accounts. The ideal architecture called for a single, centralized Sophos firewall instance to serve all VPCs, a model that dramatically reduces licensing costs, simplifies policy management, and consolidates security visibility. However, because the VPCs shared the same CIDR block (10.2.0.0/16), conventional connectivity mechanisms such as VPC Peering and AWS Transit Gateway were unable to route traffic between them.

The solution designed and implemented for this engagement introduces a dedicated Security VPC acting as a network traffic hub, fronted by a Sophos firewall performing bidirectional Network Address Translation (NAT), both source NAT (SNAT) and destination NAT (DNAT), combined with an AWS Transit Gateway attachment in the security VPC, configured in appliance mode. This architecture successfully achieves full inter-VPC communication without re-IPing any existing workloads, without application-layer changes, and without any downtime to production environments.

Key Outcomes
✔ Full bidirectional IP connectivity established between VPCs with identical CIDR blocks
✔ Single centralized Sophos firewall, eliminating the need for per-VPC firewall deployments
✔ Zero re-IPing of existing workloads or infrastructure
✔ Zero application-layer changes required across either VPC
✔ Traffic inspection, NAT translation, and firewall policy enforcement centralized
✔ Architecture validated with live ICMP and TCP traffic tests across both directions

Business & Organizational Context

The Problem: When Cloud Grows Faster Than Governance

Overlapping VPC CIDR blocks are one of the most prevalent, and most consequential, networking problems in enterprise cloud environments. While they can be avoided through disciplined IP Address Management (IPAM) from day one, they are frequently unavoidable in real-world organizations for the following structural reasons:

- **Organic, decentralized cloud adoption**
  - ❖ Engineering teams spin up VPCs independently, each defaulting to common RFC 1918 ranges such as 10.0.0.0/16 or 10.2.0.0/16, without awareness of what other teams have already allocated.
- **Mergers, acquisitions, and corporate restructuring**
  - ❖ When two organizations merge their cloud environments, it is statistically likely that their independently chosen address spaces will conflict. Neither organization had any reason to coordinate with the other prior to the merger.
- **Third-party and partner integrations**
  - ❖ Connecting to a vendor's managed environment or a partner's VPC frequently surfaces CIDR conflicts that neither party can unilaterally resolve.
- **Default VPC proliferation**

- ❖ AWS provisions a default VPC (172.31.0.0/16) in every region of every account. Organizations that neglect to replace these defaults eventually accumulate dozens of identically-ranged VPCs across accounts.

- **IaC template reuse without parameterization**
  - ❖ Terraform or CloudFormation modules copied across environments without updating CIDR variables silently propagate the same address space to every environment they provision.

Why Re-IPing Was Not a Viable Option

The theoretically straightforward solution, **simply reassigning IP addresses across one of the conflicting VPCs**, was evaluated and ruled out. In organizations with established cloud footprints, re-IPing is not a configuration change; it is a full-scale infrastructure migration project. The following factors collectively make it prohibitively risky and expensive:

- **Platform Constraint:** AWS does not support in-place CIDR modification
  - ❖ Subnets and Elastic Network Interfaces (ENIs) cannot be re-addressed in place. Every resource within the VPC must be destroyed and recreated, requiring coordinated downtime or a full parallel blue/green migration.
- **Configuration Sprawl:** IP addresses are embedded throughout application and infrastructure stacks
  - ❖ Database connection strings, environment variables, secrets managers, configuration management systems, AMI bake pipelines, and container image configurations frequently contain hardcoded IP addresses that are not visible in any single inventory.
- **Identity Infrastructure:** DNS and certificate dependencies
  - ❖ Private Route 53 hosted zones, internal PKI certificates, and Consul service mesh registries may all be bound to specific IP addresses. Updating these atomically, without a service disruption window, is extremely difficult.
- **Policy Infrastructure:** Security group and NACL rules reference CIDR blocks explicitly
  - ❖ Hundreds of security group rules across dozens of accounts may reference the old CIDR. Each must be found, validated, and updated, and any missed rule becomes a silent security or connectivity gap.

- **Infrastructure as Code:** IaC state files encode the old CIDR
  - ❖ Terraform state and CloudFormation stacks reference old CIDR values. Changing them causes plan diffs that trigger resource replacements, potentially cascading through dependent modules.
- **Systemic Risk:** Dependency discovery is open-ended
  - ❖ Every dependency fixed tends to expose additional dependencies that were previously unknown. What appears to be a three-week project routinely expands to six months as hidden consumers of the old IP space are discovered in production.

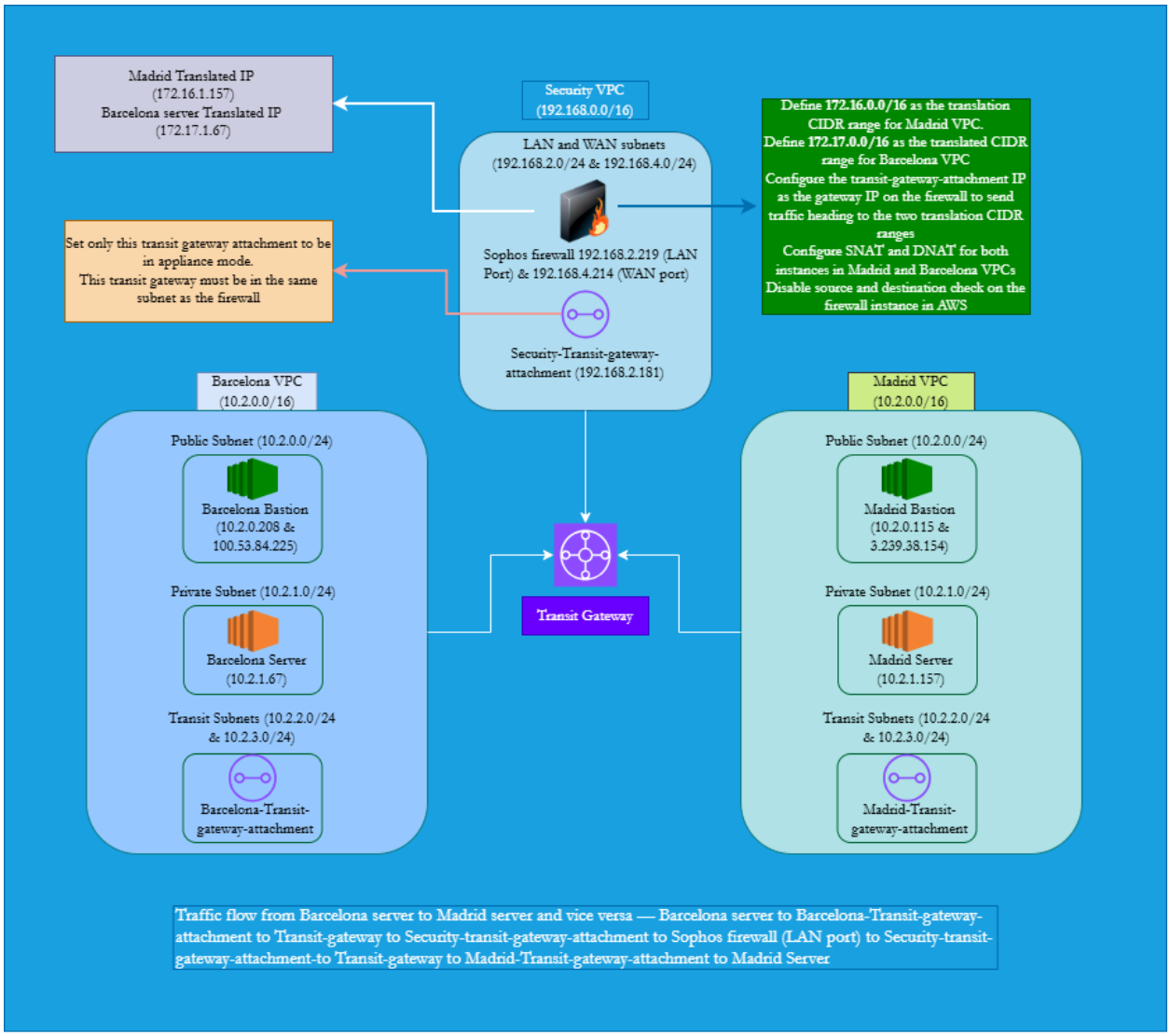
Business Impact Avoided
By implementing a NAT-based architecture instead of re-IPing, the following risks were mitigated:
• Estimated months of infrastructure migration effort across multiple teams
• Risk of partial service outages during a non-atomic CIDR migration
• Coordination overhead with application, security, networking, database, and vendor teams
• Rollback complexity in the event of a failed migration
• Potential compliance gaps during a transition window with degraded network controls

Solution Architecture

Design Philosophy

The architecture follows the principle of inserting a controlled translation boundary between the two conflicting address spaces, rather than eliminating the conflict at its source. By assigning each VPC a unique "translated" CIDR range that does not overlap with any existing address space, and configuring a central firewall to perform bidirectional NAT between the real and translated addresses, both VPCs can communicate through well-defined, routable translated identities while their actual workloads remain entirely unchanged.

This design pattern is architecturally analogous to how enterprise organizations connect to third-party networks when CIDR conflicts exist, a well-established practice in on-premises networking that this engagement extended into the AWS cloud fabric.



Component Overview

The architecture comprises four distinct network domains:

- **Spoke VPC A:** Barcelona VPC (10.2.0.0/16)
  - ❖ Source VPC. Houses production workloads including the Barcelona Server (10.2.1.67). This VPC was assigned translated CIDR range 172.17.0.0/16 for inbound identification.
- **Spoke VPC B:** Madrid VPC (10.2.0.0/16)
  - ❖ Destination VPC. Houses production workloads including the Madrid Server (10.2.1.157). This VPC was assigned translated CIDR range 172.16.0.0/16 for inbound identification.
- **Hub / Transit VPC:** Security VPC (192.168.0.0/16)

- ❖ A purpose-built VPC with no overlapping CIDR. Houses the Sophos NGFW appliance performing NAT. All inter-VPC traffic is routed through this domain.

- **Network Fabric:** AWS Transit Gateway
  - ❖ Connects all three VPCs. Configured with appliance mode enabled **exclusively on the Security VPC attachment** to ensure symmetric traffic flow through the firewall.

Traffic Flow

All communication between the Barcelona and Madrid VPCs traverses a consistent, deterministic path through the Security VPC:

1. Source server initiates a connection to the translated IP of the destination server (e.g., Barcelona Server pings 172.16.1.157, the translated address of the Madrid Server).
2. The VPC route table forwards traffic destined for the translated CIDR to the Transit Gateway.
3. The Transit Gateway, using the **spoke-to-security route table**, forwards traffic to the Security VPC's transit gateway attachment.
4. The Security VPC route table directs the translated-range traffic to the Sophos firewall's LAN port (Port A).
5. The Sophos firewall applies the NAT rule: **SNAT on the source IP to its translated equivalent**, and **DNAT on the destination IP back to its original real IP**.
6. The firewall forwards the translated packet toward the Transit Gateway through the Security attachment.
7. The Transit Gateway routes the packet to the correct spoke VPC attachment based on the original destination IP.
8. The destination server receives the packet. **The return path mirrors this process in reverse, with the firewall performing the inverse NAT translation**.

Critical Architectural Constraints

Five specific configuration requirements are non-negotiable for this architecture to function correctly. Failure to implement any one of them will result in routing loops, dropped packets, or asymmetric flows that silently break connectivity:

#	Constraint	Rationale
1	**Disable Source/Destination Check on Firewall EC2**	AWS enforces source/destination checks on all EC2 instances by default, dropping packets not addressed directly to that instance. Since the firewall must forward traffic on behalf of other hosts, this check must be disabled to allow transit traffic to be processed.
2	**Firewall and Security TGW Attachment in Same Subnet**	Static routes on the firewall must point translated CIDR traffic toward the Transit Gateway attachment IP. This is only possible if the firewall and the attachment share the same subnet, making the attachment IP a directly reachable next-hop.
3	**Transit Gateway Attachment IP as Firewall Gateway**	The firewall must be configured with the Security TGW attachment's ENI IP as its gateway IP. This enables the firewall to monitor attachment health and correctly forward return traffic.
4	**Appliance Mode Enabled Only on Security Attachment**	AWS Transit Gateway uses flow-based routing. Without appliance mode, return traffic from a spoke may traverse a different Availability Zone's attachment, bypassing the firewall and breaking stateful inspection. Appliance mode must be enabled exclusively on the Security VPC attachment.
5	**SNAT and DNAT Combined in a Single NAT Rule**	Sophos XGFW processes SNAT and DNAT as a single rule evaluation. Splitting them into separate rules causes the firewall to match only one translation per flow, resulting in asymmetric addressing that breaks bidirectional communication.

IP Address Reference

The following IP address plan was designed for this engagement. Translated CIDR ranges were selected from private address space not in use by any VPC or on-premises network in the environment.

IP Address / Range	Role / Description
10.2.0.0/16	Original CIDR, both the Barcelona and Madrid VPCs (overlapping)
10.2.1.157/32	Original IP address of the Madrid Server
10.2.1.67/32	Original IP address of the Barcelona Server
172.16.0.0/16	Translation CIDR range assigned to the Madrid VPC
172.16.1.157/32	Translated IP address of the Madrid Server
172.17.0.0/16	Translation CIDR range assigned to the Barcelona VPC
172.17.1.67/32	Translated IP address of the Barcelona Server
192.168.0.0/16	Security VPC CIDR (non-overlapping hub network)
192.168.2.181	Transit Gateway Security Attachment ENI IP (gateway for firewall)
192.168.2.219	Sophos Firewall: LAN Port (Port A) IP
192.168.4.214	Sophos Firewall: WAN Port (Port B) IP

Resource Identifiers

The following AWS resource identifiers correspond to the environment in which this architecture was deployed and tested. These are provided for reference and reproducibility.

Transit Gateway	tgw-05c754b85e673c2b7
Madrid TGW Attachment	tgw-attach-037d18c1f281fb2d4
Barcelona TGW Attachment	tgw-attach-0a378079eaa4bbc68

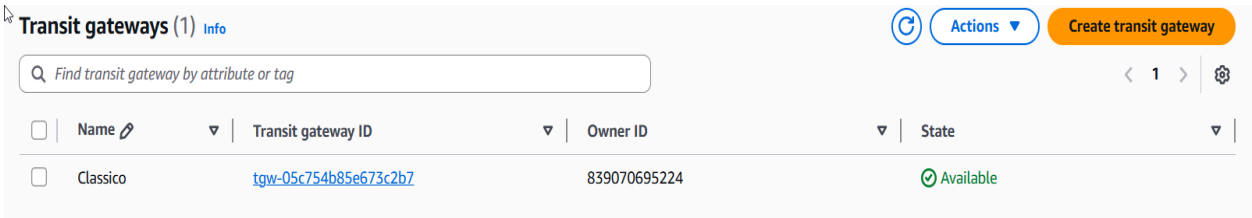
Security TGW Attachment	tgw-attach-0ea072cc5ddc0e93e
Security TGW Route Table	tgw-rtb-091185e1983ab9930 (Sec-vpc-To-Spokes)
Spoke TGW Route Table	tgw-rtb-0829f84c0b76d0fe3 (Madrid-Barcelona-To-Sec-VPC)
Barcelona VPC Route Table	rtb-0e56257d4ca90263f (Barcelona-Private-RT)
Madrid VPC Route Table	rtb-0589714fc505b73e5 (Madrid-Private-RT)
Security VPC Route Table	rtb-013f8919a1160fa8c (Sec-mgt-wan-lan-RT)
Firewall Instance	i-03bd8133af3eefe8d (PB-Sophos-Firewall)
Firewall LAN ENI	eni-0cbae5e91225be302 (Port A — 192.168.2.219)
Firewall WAN ENI	eni-0e871ca9271d14f09 (Port B — 192.168.4.214)

Implementation Guide

Phase 1: AWS Infrastructure Provisioning

Step 1 — Provision the Transit Gateway

An AWS Transit Gateway was provisioned as the central routing hub for all three VPCs. **Route table propagation was disabled on all attachments** to ensure that all routing is statically controlled, preventing the Transit Gateway from automatically learning and advertising the conflicting 10.2.0.0/16 prefix from both spokes simultaneously.



Step 2 — Create Transit Subnets in Spoke VPCs

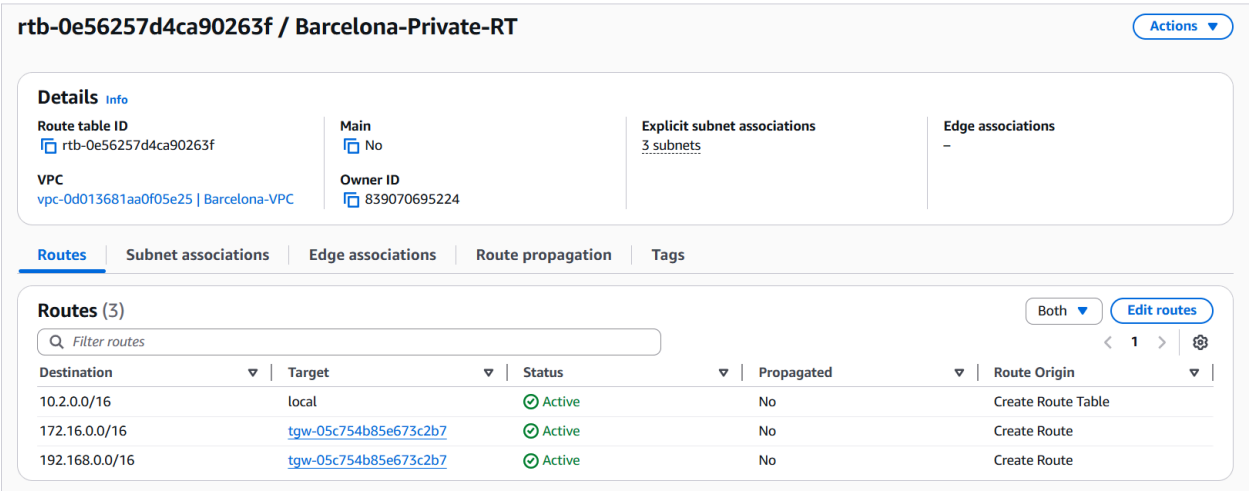
Two dedicated subnets were created in each of the overlapping VPCs (Barcelona and Madrid), spanning two Availability Zones for redundancy. These subnets were **reserved exclusively for Transit Gateway attachments**, ensuring clean separation between workload subnets and transit infrastructure.

Step 3 — Create Transit Gateway Attachments

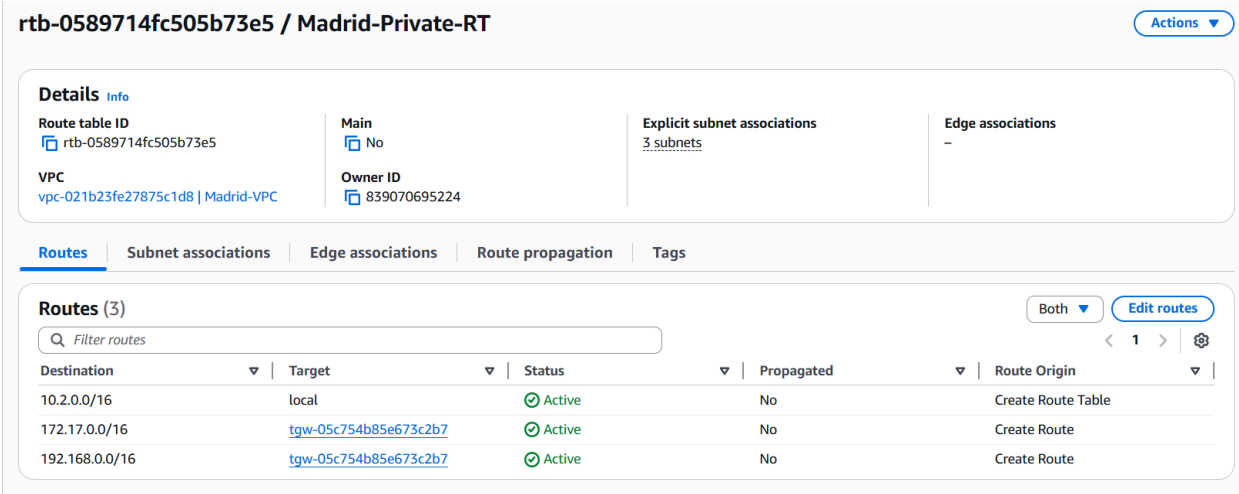
Transit Gateway attachments were created for all three VPCs. Appliance mode was explicitly disabled on the Barcelona and Madrid spoke attachments, and **explicitly enabled on the Security VPC attachment**. This asymmetric appliance mode configuration is the mechanism that ensures all flows are consistently steered through the Sophos firewall for stateful NAT processing.

Step 4 — Configure VPC Route Tables

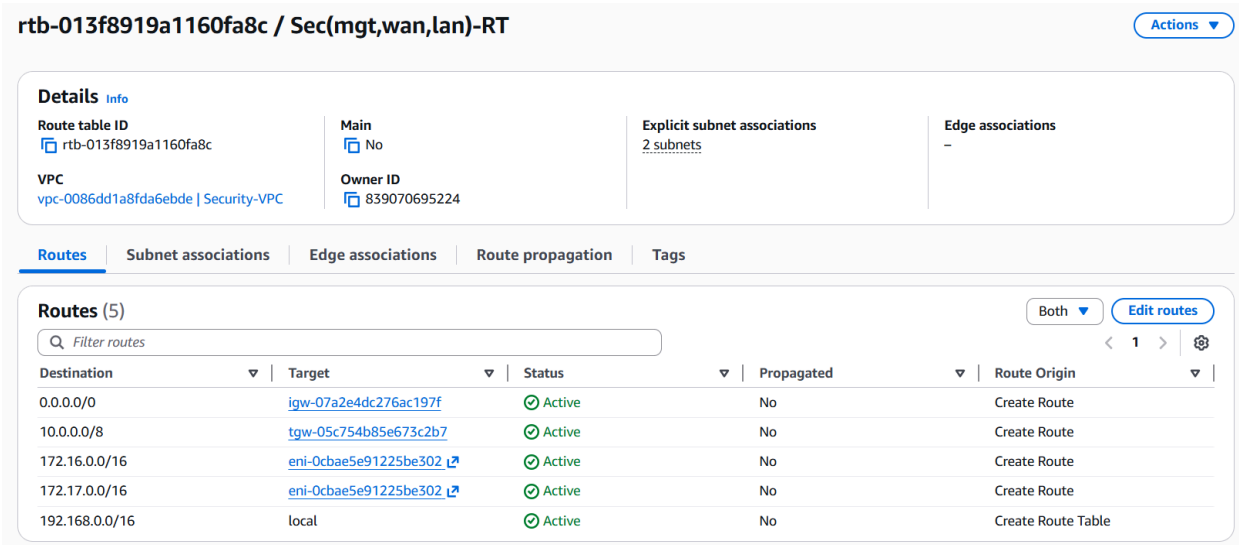
Barcelona VPC — Private Subnet Route Table: Routes 172.16.0.0/16 (Madrid translated range) and 192.168.0.0/16 (Security VPC) to the **Transit Gateway**.



Madrid VPC — Private Subnet Route Table: Routes 172.17.0.0/16 (Barcelona translated range) and 192.168.0.0/16 (Security VPC) to the **Transit Gateway**.



Security VPC — Shared Subnet Route Table: Routes 10.0.0.0/8 (covering both spoke real CIDRs) to the **Transit Gateway**; routes 172.16.0.0/16 and 172.17.0.0/16 (both translated ranges) to the **Sophos firewall's LAN ENI**; and routes 0.0.0.0/0 to the Internet Gateway for management traffic.



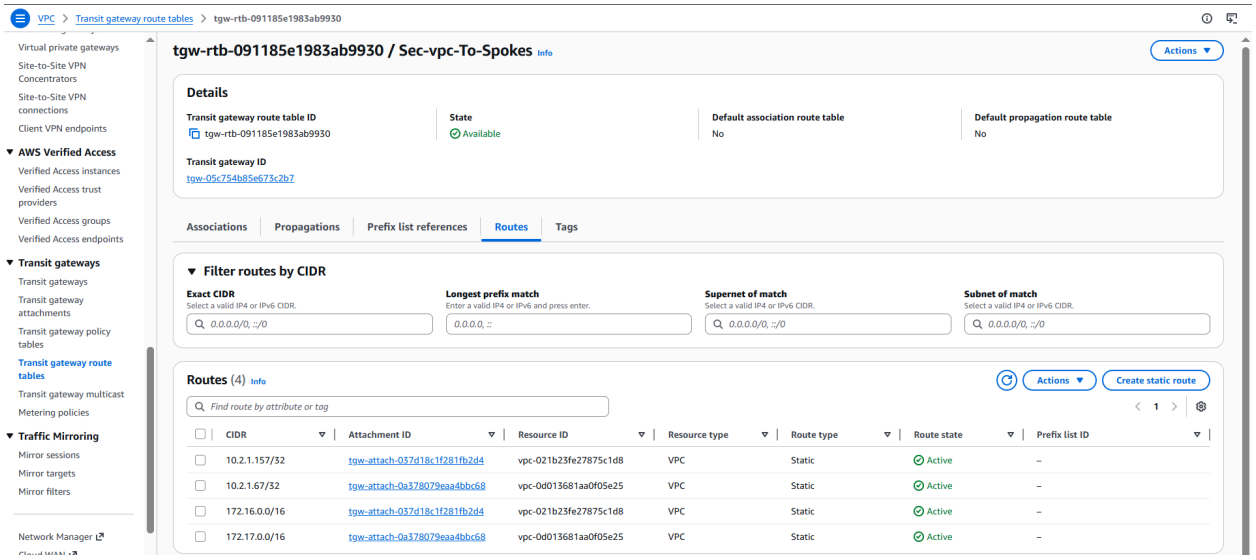
Phase 2: Transit Gateway Route Table Configuration

Security VPC Attachment Route Table (Sec-VPC-To-Spokes)

This route table governs traffic leaving the Security VPC toward the spokes. Four static routes are defined:

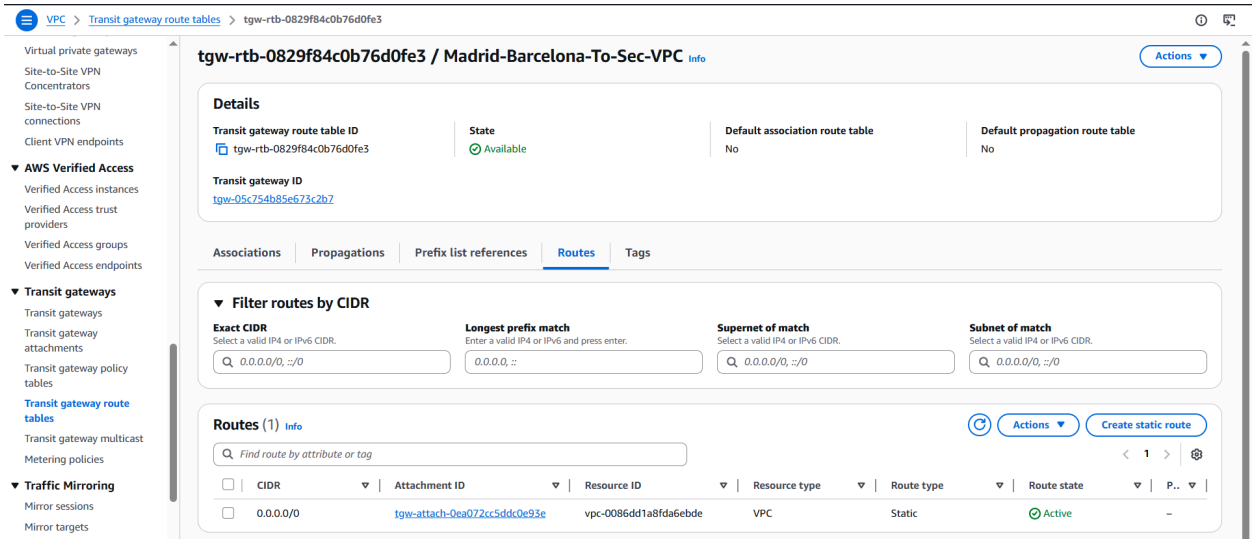
- 10.2.1.157/32 → Madrid TGW Attachment (routes to real Madrid server via spoke)

- 10.2.1.67/32 → Barcelona TGW Attachment (routes to real Barcelona server via spoke)
- 172.16.0.0/16 → Madrid TGW Attachment (routes translated Madrid range to Madrid VPC)
- 172.17.0.0/16 → Barcelona TGW Attachment (routes translated Barcelona range to Barcelona VPC)



Spoke VPC Attachment Route Table (Madrid-Barcelona-To-Sec-VPC)

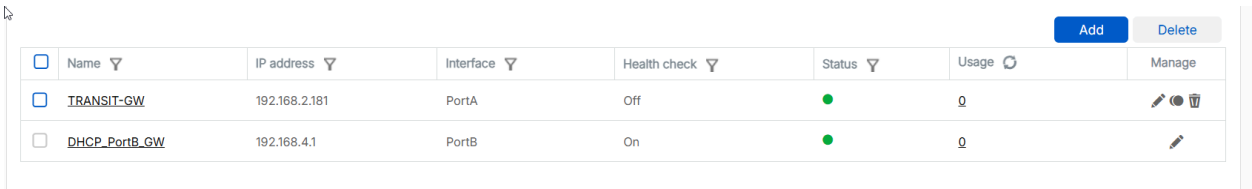
A single default route (0.0.0.0/0) pointing to the Security VPC attachment ensures that all traffic from either spoke, regardless of destination, is forwarded to the Security VPC for NAT processing by the firewall.



Phase 3: Sophos Firewall Configuration

Gateway Configuration

The Transit Gateway attachment ENI IP address (192.168.2.181) was configured as a static gateway (TRANSIT-GW) on the firewall's Port A (LAN) interface. This establishes the firewall's awareness of the TGW as a next-hop, enabling health monitoring of the attachment and correct forwarding of translated traffic toward the TGW.



Add Delete

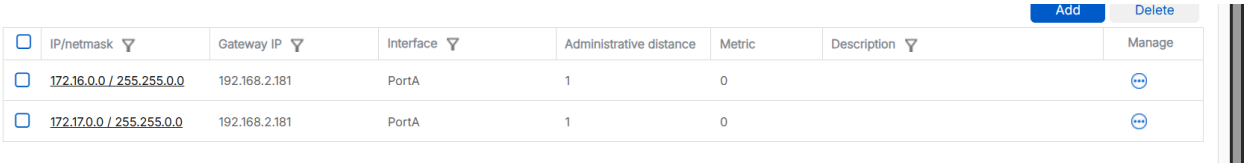
Name	IP address	Interface	Health check	Status	Usage	Manage
TRANSIT-GW	192.168.2.181	PortA	Off	●	0	
DHCP_PortB_GW	192.168.4.1	PortB	On	●	0	

Static Routes

Two static routes were added to the firewall routing table, both pointing toward the TGW gateway IP (192.168.2.181) via Port A:

- 172.16.0.0/255.255.0.0 — Translated CIDR for Madrid VPC
- 172.17.0.0/255.255.0.0 — Translated CIDR for Barcelona VPC

These routes ensure that when the firewall generates translated packets destined for the spoke VPCs, it forwards them back through the TGW rather than dropping or misrouting them.



Add Delete

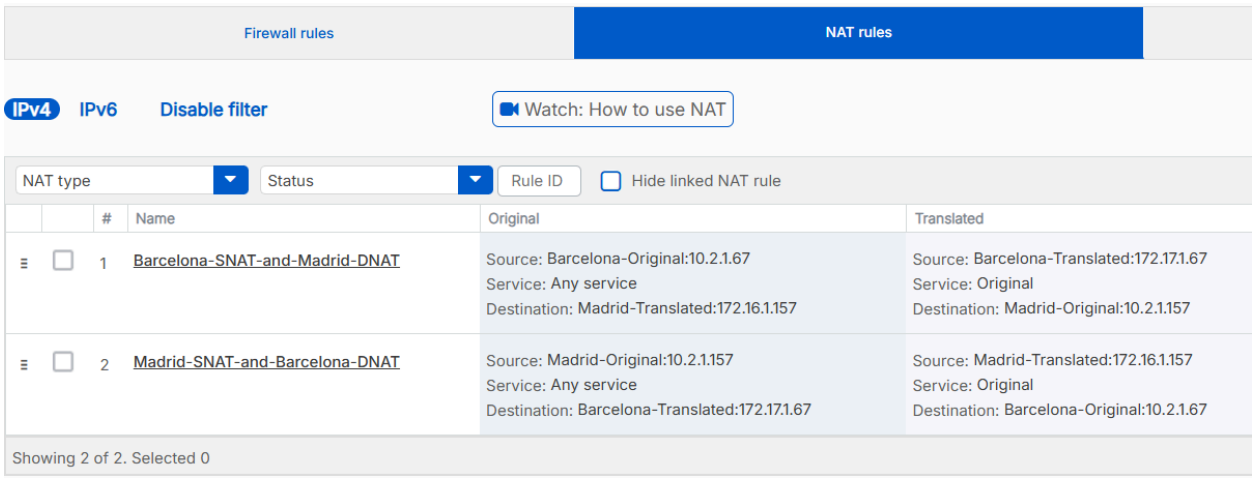
IP/netmask	Gateway IP	Interface	Administrative distance	Metric	Description	Manage
172.16.0.0 / 255.255.0.0	192.168.2.181	PortA	1	0		
172.17.0.0 / 255.255.0.0	192.168.2.181	PortA	1	0		

NAT Rules

Two bidirectional NAT rules were configured. Each rule combines SNAT and DNAT into a single policy entry, which is required by Sophos XG's NAT rule processing model:

Rule	Original (Pre-NAT)	Translated (Post-NAT)
Barcelona → Madrid	Source: 10.2.1.67 (Barcelona Server)	Source: 172.17.1.67 (Barcelona Translated IP)

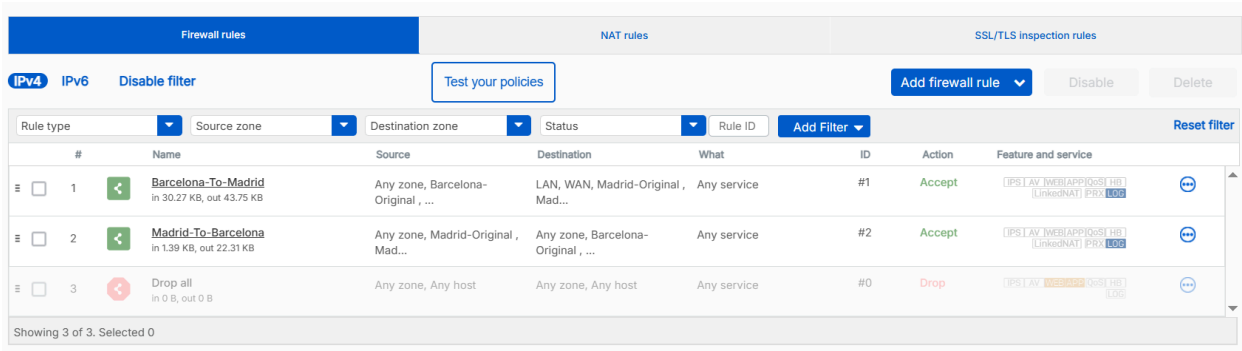
Rule	Original (Pre-NAT)	Translated (Post-NAT)
	Destination: 172.16.1.157 (Madrid Translated IP)	Destination: 10.2.1.157 (Madrid Real IP)
Madrid → Barcelona	Source: 10.2.1.157 (Madrid Server) Destination: 172.17.1.67 (Barcelona Translated IP)	Source: 172.16.1.157 (Madrid Translated IP) Destination: 10.2.1.67 (Barcelona Real IP)



Firewall Rules

Two firewall permit rules were created to authorize the inter-VPC traffic flows:

- Barcelona-To-Madrid: Permits traffic from Barcelona-Original zone/network to Madrid-Original zone/network via LAN and WAN interfaces, for any service.
- Madrid-To-Barcelona: Permits the inverse flow. A default implicit deny rule drops all other unmatched traffic.



Validation & Testing

End-to-end connectivity was validated through live traffic tests executed from within each VPC. All tests were conducted using the translated IP addresses as destinations, confirming that the NAT translation, routing, and firewall policy layers were all functioning correctly in concert.

1. Pinging from Barcelona server (10.2.1.67) to Madrid translated IP address (172.16.1.157):

```
DELL@DESKTOP-TR6H2TQ MINGW64 ~/Downloads
$ ssh -i BASTION.pem ubuntu@100.53.84.225
ubuntu@ip-10-2-1-67:~$ ping 172.16.1.157
PING 172.16.1.157 (172.16.1.157) 56(84) bytes of data.
64 bytes from 172.16.1.157: icmp_seq=1 ttl=61 time=1.55 ms
64 bytes from 172.16.1.157: icmp_seq=2 ttl=61 time=1.16 ms
64 bytes from 172.16.1.157: icmp_seq=3 ttl=61 time=1.05 ms
64 bytes from 172.16.1.157: icmp_seq=4 ttl=61 time=1.02 ms
64 bytes from 172.16.1.157: icmp_seq=5 ttl=61 time=1.03 ms
64 bytes from 172.16.1.157: icmp_seq=6 ttl=61 time=1.01 ms
64 bytes from 172.16.1.157: icmp_seq=7 ttl=61 time=1.05 ms
64 bytes from 172.16.1.157: icmp_seq=8 ttl=61 time=1.03 ms
64 bytes from 172.16.1.157: icmp_seq=9 ttl=61 time=1.03 ms
64 bytes from 172.16.1.157: icmp_seq=10 ttl=61 time=1.01 ms
64 bytes from 172.16.1.157: icmp_seq=11 ttl=61 time=1.02 ms
64 bytes from 172.16.1.157: icmp_seq=12 ttl=61 time=1.03 ms
64 bytes from 172.16.1.157: icmp_seq=13 ttl=61 time=1.18 ms
64 bytes from 172.16.1.157: icmp_seq=14 ttl=61 time=1.02 ms
64 bytes from 172.16.1.157: icmp_seq=15 ttl=61 time=1.06 ms
64 bytes from 172.16.1.157: icmp_seq=16 ttl=61 time=1.02 ms
64 bytes from 172.16.1.157: icmp_seq=17 ttl=61 time=1.03 ms
64 bytes from 172.16.1.157: icmp_seq=18 ttl=61 time=1.03 ms
64 bytes from 172.16.1.157: icmp_seq=19 ttl=61 time=1.03 ms
64 bytes from 172.16.1.157: icmp_seq=20 ttl=61 time=1.06 ms
64 bytes from 172.16.1.157: icmp_seq=21 ttl=61 time=1.23 ms
64 bytes from 172.16.1.157: icmp_seq=22 ttl=61 time=1.05 ms
64 bytes from 172.16.1.157: icmp_seq=23 ttl=61 time=1.15 ms
64 bytes from 172.16.1.157: icmp_seq=24 ttl=61 time=1.02 ms
64 bytes from 172.16.1.157: icmp_seq=25 ttl=61 time=0.998 ms
64 bytes from 172.16.1.157: icmp_seq=26 ttl=61 time=1.01 ms
64 bytes from 172.16.1.157: icmp_seq=27 ttl=61 time=1.05 ms
64 bytes from 172.16.1.157: icmp_seq=28 ttl=61 time=1.03 ms
64 bytes from 172.16.1.157: icmp_seq=29 ttl=61 time=1.04 ms
64 bytes from 172.16.1.157: icmp_seq=30 ttl=61 time=1.02 ms
64 bytes from 172.16.1.157: icmp_seq=31 ttl=61 time=0.999 ms
64 bytes from 172.16.1.157: icmp_seq=32 ttl=61 time=1.07 ms
64 bytes from 172.16.1.157: icmp_seq=33 ttl=61 time=1.03 ms
64 bytes from 172.16.1.157: icmp_seq=34 ttl=61 time=1.66 ms
64 bytes from 172.16.1.157: icmp_seq=35 ttl=61 time=1.04 ms
64 bytes from 172.16.1.157: icmp_seq=36 ttl=61 time=3.28 ms
64 bytes from 172.16.1.157: icmp_seq=37 ttl=61 time=1.02 ms
64 bytes from 172.16.1.157: icmp_seq=38 ttl=61 time=2.58 ms
64 bytes from 172.16.1.157: icmp_seq=39 ttl=61 time=1.20 ms
```

2. Pinging from Madrid server (10.2.1.157) to Barcelona translated IP address (172.17.1.67):

```
ubuntu@ip-10-2-1-157:~$ ping 172.17.1.67
PING 172.17.1.67 (172.17.1.67) 56(84) bytes of data.
64 bytes from 172.17.1.67: icmp_seq=1 ttl=61 time=1.68 ms
64 bytes from 172.17.1.67: icmp_seq=2 ttl=61 time=1.15 ms
64 bytes from 172.17.1.67: icmp_seq=3 ttl=61 time=1.10 ms
64 bytes from 172.17.1.67: icmp_seq=4 ttl=61 time=1.10 ms
64 bytes from 172.17.1.67: icmp_seq=5 ttl=61 time=1.13 ms
64 bytes from 172.17.1.67: icmp_seq=6 ttl=61 time=1.09 ms
64 bytes from 172.17.1.67: icmp_seq=7 ttl=61 time=1.19 ms
64 bytes from 172.17.1.67: icmp_seq=8 ttl=61 time=1.06 ms
64 bytes from 172.17.1.67: icmp_seq=9 ttl=61 time=1.09 ms
64 bytes from 172.17.1.67: icmp_seq=10 ttl=61 time=1.09 ms
64 bytes from 172.17.1.67: icmp_seq=11 ttl=61 time=1.10 ms
64 bytes from 172.17.1.67: icmp_seq=12 ttl=61 time=1.12 ms
64 bytes from 172.17.1.67: icmp_seq=13 ttl=61 time=1.07 ms
64 bytes from 172.17.1.67: icmp_seq=14 ttl=61 time=1.10 ms
64 bytes from 172.17.1.67: icmp_seq=15 ttl=61 time=1.11 ms
64 bytes from 172.17.1.67: icmp_seq=16 ttl=61 time=1.18 ms
64 bytes from 172.17.1.67: icmp_seq=17 ttl=61 time=1.07 ms
```

3. Firewall applying firewall and NAT rules, as well as logging the traffic:

Firewall | Search...

Filter: No filter active | Add filter | Timer filter | Reset

	Time	Log comp	Log subtype	Username	Firewall rule	Firewall rule name	NAT rule	NAT rule name	In interface	Out interface	Src IP	Dst IP	Src port	Dst port	Protocol	Rule type	Live
Firewall	2026-03-25 22:10:19	Firewall Rule	Allowed		2	Madrid-To-Barcel...	4	Madrid-SNAT	PortA	PortB	10.2.1.157	172.17.1.67			ICMP	1	Ope PCA
Firewall	2026-03-25 22:08:03	Firewall Rule	Allowed		1	Barcelona-To-Mad...	3	Barcelona-SNAT	PortA	PortB	10.2.1.67	172.16.1.157			ICMP	1	Ope PCA
Firewall	2026-03-25 21:55:36	Firewall Rule	Allowed		1	Barcelona-To-Mad...	3	Barcelona-SNAT	PortA	PortB	10.2.1.67	172.16.1.157			ICMP	1	Ope PCA
Firewall	2026-03-25 21:54:20	Firewall Rule	Allowed		1	Barcelona-To-Mad...	3	Barcelona-SNAT	PortA	PortB	10.2.1.67	172.16.1.157			ICMP	1	Ope PCA
Firewall	2026-03-25 21:52:36	Firewall Rule	Allowed		1	Barcelona-To-Mad...	3	Barcelona-SNAT	PortA	PortB	10.2.1.67	172.16.1.157	44216	80	TCP	1	Ope PCA
Firewall	2026-03-25 21:52:29	Firewall Rule	Allowed		1	Barcelona-To-Mad...	3	Barcelona-SNAT	PortA	PortB	10.2.1.67	172.16.1.157	44998	80	TCP	1	Ope PCA

Test	Destination	Protocol	Result
Barcelona → Madrid	172.16.1.157 (Madrid Translated IP)	ICMP	✅ **Success**
Madrid → Barcelona	172.17.1.67 (Barcelona Translated IP)	ICMP	✅ **Success**
Firewall Log Verification	Both directions	ICMP / TCP	✅ **NAT Applied**

Firewall logs confirmed that the correct NAT rules and firewall rules were being matched for each flow, with both SNAT and DNAT operations applied as expected. ICMP round-trip times were consistently sub-2ms, indicating no measurable latency overhead introduced by the NAT layer.

Skills & Competencies Demonstrated

This engagement demonstrates a broad set of cloud networking, security engineering, and solution architecture competencies applicable to senior cloud infrastructure roles:

Competency Area	Evidence from this Engagement
AWS Advanced Networking	Designed and implemented a multi-VPC Transit Gateway topology with custom route tables, appliance mode, and spoke-to-hub routing policies.
Network Security Engineering	Deployed and configured a Sophos NGFW in a cloud environment with bidirectional NAT policies, static routing, firewall rules, and gateway health monitoring.
Problem Diagnosis & Root Cause Analysis	Identified the architectural root cause of CIDR overlap, evaluated re-IPing versus NAT-based remediation, and selected the lowest-risk path.
Solution Architecture	Designed a hub-and-spoke Security VPC architecture from first principles to accommodate an overlapping address space without disrupting existing workloads.
Cost Optimization	Avoided per-VPC firewall deployments by centralizing the Sophos appliance, reducing licensing and operational overhead to a single instance.
Technical Documentation	Produced end-to-end architecture documentation covering design rationale, implementation steps, configuration tables, and validation evidence.
Zero Trust & ZTNA	Architected the network foundation required for a Sophos ZTNA deployment across a multi-VPC environment with conflicting address spaces.

Lessons Learned & Recommendations

For Organizations Facing Overlapping CIDRs

- Evaluate re-IPing cost honestly before committing to it
  - ❖ Most organizations significantly underestimate re-IPing scope. A NAT-based hub architecture typically delivers faster time-to-connectivity and lower risk.
- Use AWS IPAM proactively
  - ❖ AWS IP Address Manager (IPAM) provides centralized visibility and allocation governance across all accounts and regions. Implementing it before conflicts arise is dramatically less expensive than remediation.
- Treat CIDR allocation as organizational policy, not team preference
  - ❖ CIDR allocation should be governed centrally, with address ranges pre-allocated to accounts and regions before teams begin provisioning.

Architectural Nuances to Preserve in Future Deployments

- **Appliance mode asymmetry is intentional and must not be changed**
  - ❖ Enabling appliance mode on spoke attachments will break flow symmetry. Only the Security VPC attachment should have it enabled.
- **The SNAT+DNAT single-rule requirement is Sophos-specific**
  - ❖ Other firewall vendors may allow separate SNAT and DNAT rules. In Sophos XG, they must be combined. This is a platform-specific nuance that must be documented for future maintainers.
- **Source/destination check disablement is persistent but not automatic**
  - ❖ If the firewall instance is replaced, the source/destination check must be explicitly disabled on the new instance. This is not inherited from AMIs or launch templates.

This document was produced as an original technical case study.

All architecture, configuration, and validation work was designed and executed independently.